3.1 Composable 런타임 구성 분석

개요

Composable 런타임은 Polkadot 파라체인으로 동작하는 블록체인의 핵심 구성 요소입니다. 이 문서에서는 `construct_runtime!` 매크로를 통한 팰릿 통합 과정, `Runtime` 구조체와 각 팰릿의 `Config` 트레이트 구현, 그리고 제네시스 설정 및 체인 스펙 구성을 상세히 분석합니다.

3.1.1 construct_runtime! 매크로를 통한 팰릿 통합

매크로 구조 분석

`construct_runtime!` 매크로는 Substrate 프레임워크의 핵심 매크로로, 여러 팰릿들을 하나의 런타임으로 통합하는 역할을 합니다.

```
// code/parachain/runtime/composable/src/lib.rs

construct_runtime!(

pub  enum Runtime where

Block = Block,

NodeBlock = opaque::Block,

UncheckedExtrinsic = UncheckedExtrinsic

{

// 시스템 레벨 팰릿들

System: system = 0,

Timestamp: timestamp = 1,
```

```
Sudo: sudo = 2,

TransactionPayment: transaction_payment = 4,

AssetTxPayment : asset_tx_payment = 12,

Indices: indices = 5,

Balances: balances = 6,

Multisig: multisig = 8,

// 파라체인 관련 팰릿들

ParachainSystem: cumulus_pallet_parachain_system = 10,

ParachainInfo: parachain_info = 11,

// 콜레이터 지원 팰릿들 (순서 중요)

Authorship: authorship = 20,

CollatorSelection: collator_selection = 21,

Session: session = 22,

Aura: aura = 23,

AuraExt: cumulus_pallet_aura_ext = 24,

// 거버넌스 유틸리티들

Council: collective::<Instance1> = 30,

CouncilMembership: membership::<Instance1> = 31,

Treasury: treasury::<Instance1> = 32,

Democracy: democracy = 33,

TechnicalCommittee: collective::<Instance2> = 72,

TechnicalCommitteeMembership: membership::<Instance2> = 73,

// XCM 헬퍼들
```

```
XcmpQueue: cumulus_pallet_xcmp_queue = 40,

PolkadotXcm: pallet_xcm = 41,

CumulusXcm: cumulus_pallet_xcm = 42,

DmpQueue: cumulus_pallet_dmp_queue = 43,

XTokens: orml_xtokens = 44,

UnknownTokens: orml_unknown_tokens = 45,

// 자산 관리 팰릿들

Tokens: orml_tokens = 52,

Assets: pallet_assets = 57,

AssetsRegistry: assets_registry = 59,

// IBC 및 크로스체인 팰릿들

Ibc: pallet_ibc = 190,

Ics20Fee: pallet_ibc::ics20_fee = 191,

PalletMultihopXcmIbc: pallet_multihop_xcm_ibc = 192,

PalletXcmHelper: pallet_xcm_helper = 193,

PalletLiquidStaking: pallet_liquid_staking = 194,

}

);
```

팰릿 인덱스 체계

각 팰릿에는 고유한 인덱스 번호가 할당되며, 이는 다음과 같은 규칙을 따릅니다:

- **0-9**: 핵심 시스템 팰릿들

- **10-19**: 파라체인 관련 팰릿들
- **20-29**: 콜레이터 및 컨센서스 팰릿들
- **30-39**: 거버넌스 관련 팰릿들
- **40-49**: XCM 및 크로스체인 통신 팰릿들
- **50-59**: 자산 관리 팰릿들
- **190-199**: IBC 및 고급 크로스체인 기능 팰릿들

매크로 생성 코드

`construct_runtime!` 매크로는 다음과 같은 코드를 자동 생성합니다:

1. **Runtime 열거형**: 모든 팰릿을 포함하는 런타임 구조체
2. **PalletInfo**: 팰릿 메타데이터 제공
3. **AllPalletsWithSystem**: 모든 팰릿의 집합
4. **RuntimeCall**: 모든 팰릿의 호출을 통합하는 열거형
5. **RuntimeEvent**: 모든 팰릿의 이벤트를 통합하는 열거형

3.1.2 Runtime 구조체와 Config 트레이트 구현

System 팰릿 구성

System 팰릿은 모든 Substrate 런타임의 기본 팰릿으로, 블록체인의 기본 기능을 제공합니다:

```
impl system::Config for Runtime {
type BaseCallFilter = BaseCallFilter;
type BlockWeights = RuntimeBlockWeights;
type BlockLength = RuntimeBlockLength;
```

```
type AccountId = AccountId;

type RuntimeCall = RuntimeCall;

type Lookup = AccountIdLookup<AccountId, AccountIndex>;

type Index = AccountIndex;

type BlockNumber = BlockNumber;

type Hash = Hash;

type Hashing = BlakeTwo256;

type Header = generic::Header<BlockNumber, BlakeTwo256>;

type RuntimeEvent = RuntimeEvent;

type RuntimeOrigin = RuntimeOrigin;

type BlockHashCount = BlockHashCount;

type DbWeight = RocksDbWeight;

type Version = Version;

type AccountData = balances::AccountData<Balance>;

type PalletInfo = PalletInfo;

type OnNewAccount = ();

type OnKilledAccount = ();

type SystemWeightInfo = weights::frame_system::SubstrateWeight<Runtime>;

type SS58Prefix = SS58Prefix;

type OnSetCode = cumulus_pallet_parachain_system::ParachainSetCode<Self>;

type MaxConsumers = frame_support::traits::ConstU32<16>;

}
```

파라체인 시스템 구성

파라체인으로 동작하기 위한 핵심 설정:

```
impl cumulus_pallet_parachain_system::Config for Runtime {
type RuntimeEvent = RuntimeEvent;
type OnSystemEvent = ();
type SelfParaId = parachain_info::Pallet<Runtime>;
type OutboundXcmpMessageSource = XcmpQueue;
type DmpMessageHandler = DmpQueue;
type ReservedDmpWeight = ReservedDmpWeight;
type XcmpMessageHandler = XcmpQueue;
type ReservedXcmpWeight = ReservedXcmpWeight;
type CheckAssociatedRelayNumber = cumulus_pallet_parachain_system::RelayNumberSt
}
```

자산 레지스트리 구성

다중 자산 지원을 위한 자산 레지스트리 설정:

```
impl assets_registry::Config for Runtime {
type RuntimeEvent = RuntimeEvent;
type LocalAssetId = CurrencyId;
type Balance = Balance;
type ForeignAssetId = ForeignAssetId;
```

```
type UpdateAssetRegistryOrigin = EnsureRootOrHalfCouncil;

type ParachainOrGovernanceOrigin = EnsureRootOrHalfCouncil;

type WeightInfo = weights::assets_registry::WeightInfo<Runtime>;

type Convert = sp_runtime::traits::ConvertInto;

type NetworkId = ComposableNetworkId;

}
```

Liquid Staking 팰릿 구성

유동성 스테이킹 기능을 위한 복잡한 구성:

```
impl pallet_liquid_staking::Config for Runtime {

type RuntimeEvent = RuntimeEvent;

type RuntimeOrigin = RuntimeOrigin;

type RuntimeCall = RuntimeCall;

type PalletId = StakingPalletId;

type WeightInfo = pallet_liquid_staking::weights::SubstrateWeight<Runtime>;

type SelfParaId = ParachainInfo;

type Assets = Assets;

type RelayOrigin = EnsureRoot<AccountId>;

type UpdateOrigin = EnsureRootOrHalfCouncil;

type DerivativeIndexList = DerivativeIndexList;

type XcmFees = XcmFees;

type MatchingPoolFastUnstakeFee = MatchingPoolFastUnstakeFee;
```

```
type DistributionStrategy = pallet_liquid_staking::distribution::MaxMinDistribut

type StakingCurrency = StakingCurrency;

type LiquidCurrency = LiquidCurrency;

type EraLength = EraLength;

type MinStake = MinStakeLSD;

type MinUnstake = MinUnstake;

type XCM = PalletXcmHelper;

type BondingDuration = BondingDuration;

type MinNominatorBond = MinNominatorBond;

type RelayChainValidationDataProvider = RelayChainValidationDataProvider<Runtime

type Members = Members<AccountId>;

type NumSlashingSpans = NumSlashingSpans;

type ElectionSolutionStoredOffset = ElectionSolutionStoredOffset;

type ProtocolFeeReceiver = LsdProtocolFeeAccount;

type Decimal = Decimal;

type NativeCurrency = NativeAssetId;

}
```

3.1.3 제네시스 설정 및 체인 스펙 구성

제네시스 구성 구조

제네시스 블록은 블록체인의 초기 상태를 정의합니다:

```
// code/parachain/node/src/chain_spec/composable.rs

pub fn genesis_config(

root: AccountId,

invulnerables: Vec<(AccountId, AuraId)>,

accounts: Vec<AccountId>,

id: ParaId,

existential_deposit: Balance,

treasury: AccountId,

) -> composable_runtime::GenesisConfig {

composable_runtime::GenesisConfig {

system: composable_runtime::SystemConfig {

code: composable_runtime::WASM_BINARY_V2

.expect("WASM binary was not build, please build it!")

.to_vec(),

},

balances: composable_runtime::BalancesConfig {

balances: vec![

vec![(treasury, existential_deposit)],

accounts.iter().cloned().map(|k| (k, 1 << 60)).collect(),

]

.concat(),

},
```

```
open_gov_balances: Default::default(),

aura: Default::default(),

sudo: composable_runtime::SudoConfig {

key: Some(root.clone())

},

indices: composable_runtime::IndicesConfig {

indices: vec![]

},

parachain_info: composable_runtime::ParachainInfoConfig {

parachain_id: id

},

session: composable_runtime::SessionConfig {

keys: invulnerables

.iter()

.cloned()

.map(|(acc, aura)| {

(

acc.clone(), // account id

acc, // validator id

session_keys(aura), // session keys

)

})

.collect(),
```

```
},
collator_selection: composable_runtime::CollatorSelectionConfig {
invulnerables: invulnerables.iter().cloned().map(|(acc, _)| acc).collect(),
candidacy_bond: existential_deposit * 16,
..Default::default()
},
assets_registry: composable_runtime::AssetsRegistryConfig {
assets: primitives::topology::Composable::assets(),
phantom: Default::default(),
},
ibc: composable_runtime::IbcConfig {
assets: vec![pallet_ibc::pallet::AssetConfig {
id: primitives::currency::CurrencyId::COMPOSABLE_LAYR,
denom: b"1".to_vec(),
}],
},
// 기타 팰릿들의 기본 설정
technical_committee_membership: composable_runtime::TechnicalCommitteeMembership
members: vec![root.clone()].try_into().expect("const"),
phantom: Default::default(),
},
release_membership: composable_runtime::ReleaseMembershipConfig {
members: vec![root].try_into().expect("const"),
```

```
phantom: Default::default(),
},
pallet_liquid_staking: Default::default(),
}
}
```

체인 스펙 구성

체인 스펙은 네트워크별 설정을 정의합니다:

```
// 개발 환경용 체인 스펙
pub fn composable_dev() -> composable::ChainSpec {
let mut properties = Properties::new();
properties.insert("tokenSymbol".into(), "LAYR".into());
properties.insert("tokenDecimals".into(), CurrencyId::decimals().into());
properties.insert("ss58Format".into(), composable_runtime::SS58Prefix::get().int

composable::ChainSpec::from_genesis(
"Local Composable Testnet",
"composable",
ChainType::Development,
move || {
composable::genesis_config(
account_id_from_seed::<sr25519::Public>("Alice"),
```

```
vec![
(
account_id_from_seed::<sr25519::Public>("Alice"),
get_collator_keys_from_seed("Alice"),
),
(
account_id_from_seed::<sr25519::Public>("Bob"),
get_collator_keys_from_seed("Bob"),
),
],
dev_accounts(),
*PARACHAIN_ID,
composable_runtime::ExistentialDeposit::get(),
composable_runtime::TreasuryAccount::get(),
)
},
vec![],
None,
None,
None,
Some(properties),
Extensions {
relay_chain: "westend_local_testnet".into(),
```

```
para_id: u32::from(*PARACHAIN_ID),
},
)
}
```

파라체인 확장 설정

파라체인 특화 설정을 위한 확장 구조체:

```
#[derive(
Debug, Clone, PartialEq, Eq, Serialize, Deserialize, ChainSpecGroup, ChainSpecEx
)]
#[serde(deny_unknown_fields)]
pub struct Extensions {
/// 파라체인의 릴레이 체인
pub relay_chain: String,
/// 파라체인의 ID
pub para_id: u32,
}
```

3.1.4 런타임 매개변수 설정

블록 제한 및 가중치

```
parameter_types! {
pub  const BlockHashCount: BlockNumber = 250;
pub RuntimeBlockLength: BlockLength =
BlockLength::max_with_normal_ratio(5 * 1024 * 1024, NORMAL_DISPATCH_RATIO);
pub RuntimeBlockWeights: BlockWeights = BlockWeights::builder()
.base_block(BlockExecutionWeight::get())
.for_class(DispatchClass::all(), |weights| {
weights.base_extrinsic = ExtrinsicBaseWeight::get();
})
.for_class(DispatchClass::Normal, |weights| {
weights.max_total = Some(NORMAL_DISPATCH_RATIO * MAXIMUM_BLOCK_WEIGHT);
})
.for_class(DispatchClass::Operational, |weights| {
weights.max_total = Some(MAXIMUM_BLOCK_WEIGHT);
weights.reserved = Some(
MAXIMUM_BLOCK_WEIGHT - NORMAL_DISPATCH_RATIO * MAXIMUM_BLOCK_WEIGHT
);
})
.avg_block_initialization(AVERAGE_ON_INITIALIZE_RATIO)
.build_or_panic();
pub  const SS58Prefix: u8 = 50;
pub  const ComposableNetworkId: u32 = 1;
```

```
}
```

XCM 관련 매개변수

```
parameter_types! {
/// XCMP를 위해 블록 가중치의 1/4 예약
pub const ReservedXcmpWeight: Weight = MAXIMUM_BLOCK_WEIGHT.saturating_div(4);
/// 다운워드 메시지 처리를 위해 블록 가중치의 1/4 예약
pub const ReservedDmpWeight: Weight = MAXIMUM_BLOCK_WEIGHT.saturating_div(4);
}
```

3.1.5 런타임 API 구현

런타임은 다양한 API를 구현하여 외부 클라이언트와 상호작용합니다:

```
impl_runtime_apis! {
impl assets_runtime_api::AssetsRuntimeApi<Block, CurrencyId, AccountId, Balance,
fn balance_of(SafeRpcWrapper(asset_id): SafeRpcWrapper<CurrencyId>, account_id:
SafeRpcWrapper(<Assets as frame_support::traits::fungibles::Inspect::<AccountId>
}

fn list_assets() -> Vec<Asset<SafeRpcWrapper<u128>, SafeRpcWrapper<Balance>, Fo
let all_assets = assets_registry::Pallet::<Runtime>::get_all_assets();
all_assets.iter().map(|asset|
```

```
Asset {
decimals : asset.decimals,
existential_deposit : SafeRpcWrapper(multi_existential_deposits::<AssetsRegistry
id : SafeRpcWrapper(asset.id.into()),
foreign_id : asset.foreign_id.clone(),
name : asset.name.clone(),
symbol : asset.symbol.clone(),
ratio : asset.ratio,
}
).collect()
}
}

impl sp_api::Core<Block> for Runtime {
fn  version() -> RuntimeVersion {
VERSION
}

fn  execute_block(block: Block) {
Executive::execute_block(block);
}

fn  initialize_block(header: &<Block as BlockT>::Header) {
```

```
Executive::initialize_block(header)

}

}

}
```

결론

Composable 런타임은 `construct_runtime!` 매크로를 통해 50개 이상의 팰릿을 체계적으로 통합하여 복잡한 DeFi 및 크로스체인 기능을 제공합니다. 각 팰릿의 `Config` 트레이트 구현을 통해 런타임별 특화 설정이 가능하며, 제네시스 설정과 체인 스펙을 통해 다양한 네트워크 환경에서 동작할 수 있도록 구성되어 있습니다.

이러한 구조는 Substrate 프레임워크의 모듈성과 확장성을 잘 보여주며, 파라체인으로서의 특별한 요구사항들을 효과적으로 처리할 수 있도록 설계되었습니다.